

RSA

Claus Fieker

7. Juni 2024

Für Zahlen a und N sei $a \bmod N$ der Rest bei der Division von a durch N .

Also $15 \bmod 7 = 1$, $0 \bmod 11 = 0$, $7 \bmod 7 = 0$.

Damit kann ganz „normal“ gerechnet werden:

$$((a \bmod N) + (b \bmod N)) \bmod N = (a + b) \bmod N$$

$$((a \bmod N) \cdot (b \bmod N)) \bmod N = (a \cdot b) \bmod N$$

Zum Beispiel $(7 \cdot 7) \bmod 5 = (2 \cdot 2) \bmod 5 = 4$

Denn:

- ▶ $a \bmod N = (a + sN) \bmod N$ für jedes s
- ▶ $a = q_a N + (a \bmod N)$ (Division mit Rest, q_a ist der Quotient)

$$\begin{aligned}(ab) \bmod N &= \left(\underbrace{(q_a N + (a \bmod N))}_a \underbrace{(q_b N + (b \bmod N))}_b \right) \bmod N \\&= \underbrace{(q_a N)(q_b N) + q_a N(b \bmod N) + q_b N(a \bmod N)}_{=(q_a q_b N + q_a(b \bmod N) + q_b(a \bmod N))N} \\&\quad + (a \bmod N)(b \bmod N) \bmod N \\&= (a \bmod N)(b \bmod N) \bmod N\end{aligned}$$

Ebenso für $a + b$.

Damit können wir in langen Rechnungen immer mal wieder „den Rest“ nehmen um Zahlen klein zu halten.

Zusätzlich:

$$a \bmod N = 0 \iff N \text{ teilt } a$$

Beispiel: $2^{64} \bmod 7$:

$$2^{64} = (2^4)^{16}$$

Also

$$2^{64} \bmod 7 = (2^4 \bmod 7)^{16} \bmod 7 = 2^{16} \bmod 7$$

(da $2^4 = 16$ und $16 \bmod 7 = 2$). Und nochmals

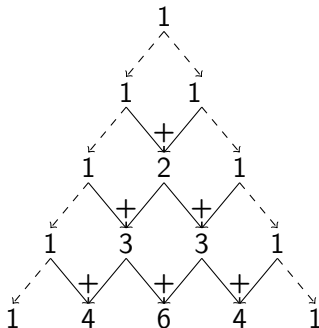
$$2^{16} = (2^4)^4$$

und somit

$$2^{16} \bmod 7 = (2^4 \bmod 7)^4 \bmod 7 = 2^4 \bmod 7 = 2$$

(Was einfacher ist als $2^{64} = 18446744073709551616$ zu berechnen und dann durch 7 zu teilen.

Pascalsches Dreieck



Wir setzen

- ▶ $n! = 1 \cdot 2 \cdot \dots \cdot (n-1) \cdot n$ „ n -Fakultät“
- ▶ $\binom{n}{k} = \frac{n!}{k!(n-k)!} = \frac{n \cdot (n-1) \cdot \dots \cdot (k+1)}{1 \cdot 2 \cdot \dots \cdot (n-k)}$ „ n über k “

Dann ist $\binom{n}{k}$ genau der k -Eintrag der n -ten Zeile des Pascalschen Dreiecks!, Denn:

$$\begin{aligned}\binom{n}{k} + \binom{n}{k+1} &= \frac{n!}{k!(n-k)!} + \frac{n!}{(k+1)!(n-(k+1))!} \\ &= \frac{n!}{k!(n-k)!} + \frac{n!(n-k)}{(k+1)k!(n-k)!} \\ &= \frac{n!(k+1)}{(k+1)k!(n-k)!} + \frac{n!(n-k)}{(k+1)k!(n-k)!} \\ &= \frac{n!(k+1) + n!(n-k)}{(k+1)!((n+1)-(k+1))!} = \binom{n+1}{k}\end{aligned}$$

$$\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k}$$

Damit (und mit $\binom{0}{0} = \binom{n}{0} = \binom{n}{n} = 1$) erfüllt $\binom{n}{k}$ die selben Rechenregeln wie die Einträge im Pascalschen Dreieck.

Damit können wir die Binomischen Formeln erweitern:

$$(a+b)^n = \sum_{k=0}^n \binom{n}{k} a^k b^{n-k}$$

Denn

$$\begin{aligned}(a+b)^{n+1} &= (a+b)^n(a+b) = a(a+b)^n + b(a+b)^n \\&= \sum_{k=0}^n \binom{n}{k} a^{k+1} b^{n-k} + \sum_{k=0}^n \binom{n}{k} a^k b^{n+1-k} \\&= a^{n+1} + b^{n+1} + \sum_{k=1}^n \left(\binom{n}{k} + \binom{n}{k-1} \right) a^k b^{n+1-k}\end{aligned}$$

$$(a + b)^2 = a^2 + 2ab + b^2$$

$$(a + b)^3 = (a^2 + 2ab + b^2)(a + b) = a^3 + 3a^2b + 3ab^2 + b^3$$

$$(a+b)^4 = (a^3+3a^2b+3ab^2+b^3)(a+b) = a^4+4a^3b+6a^2b^2+4ab^3+b^4$$

...

Warum??

Weil ich das jetzt mit modulo Rechnen verbinden will!

Sei nun p eine Primzahl, dann gilt für $1 \leq k < p$: $\binom{p}{k} \bmod p = 0$.

Denn $\binom{p}{k} = \frac{p!}{k!(p-k)!}$. Da im Pascalschen Dreieck nur ganze Zahlen stehen, so ist $\binom{p}{k}$ auch ganz. Im Zähler steht immer ein p da $k < p$ und $p - k < p$ gilt, also ist es immer durch p teilbar, daher ist der Rest 0.

Damit erhalten wir für Primzahlen viel schönere Binomische Formeln:

$$(a + b)^p \bmod p = (a^p + b^p) \bmod p$$

Fermat

Für jede Primzahl p und $0 \leq a < p$ gilt:

$$a^p \bmod p = a$$

Denn: $0^p = 0$, $1^p = 1$ und mit Induktion: falls $a^p \bmod p = a$, dann auch

$$(a + 1)^p \bmod p = (a^p + 1) \bmod p = a + 1.$$

Cooler Anwendung auf dem Rechner: falls $a^N \bmod N \neq a$, dann kann N keine Primzahl sein! Dies kann schnell getestet werden!

Euklid

Seien $a > b > 0$ natürliche Zahlen, dann gilt:

$$\text{ggT}(a, b) = \text{ggT}(b, a \bmod b)$$

denn, mit $g = \text{ggT}(a, b)$:

- ▶ $a = qb + (a \bmod b)$
- ▶ $a = gx, b = gy$
- ▶ also $gx = q(gy) + (a \bmod b)$
- ▶ also $a \bmod b = gx - qgy = g(x - qy)$ ist auch durch g teilbar.
- ▶ genauso: alles was b und $a \bmod b$ teilt, teilt auch a .

Dies ist *schnell* (auf dem Rechner)

(Falls $a \bmod b = 0$, so ist $b = \text{ggT}(a, b)$)

Bezout

... aber es geht noch mehr! Es gibt x, y mit

$$g = \text{ggT}(a, b) = xa + yb$$

der ggT ist eine „Vielfachsumme“

Der erweiterte euklidische Algorithmus!

Idee: in jedem Schritt oben können wir nachhalten, wie sich die Zahlen ändern - und wie sie sich aus dem Anfang zusammensetzen.

Bezout - Beispiel

$$\begin{array}{lcl}
 \text{ggT}(8, 5) = & \left| \begin{array}{l} 8 = 1 \cdot 8 + 0 \cdot 5 \\ 3 = 8 - 5 \end{array} \right| & \left| \begin{array}{l} 5 = 0 \cdot 8 + 1 \cdot 5 \end{array} \right. \\
 \text{ggT}(5, 3) = & \left| \begin{array}{l} 5 = 0 \cdot 8 + 1 \cdot 5 \\ 2 = 5 - 3 \end{array} \right| & \left| \begin{array}{l} 3 = 1 \cdot 8 - 1 \cdot 5 \end{array} \right. \\
 \text{ggT}(3, 2) = & \left| \begin{array}{l} 3 = 1 \cdot 8 - 1 \cdot 5 \\ 1 = 3 - 2 \end{array} \right| & \left| \begin{array}{l} 2 = -1 \cdot 8 + 2 \cdot 5 \end{array} \right. \\
 \text{ggT}(2, 1) = 1 & \left| \begin{array}{l} 2 = -1 \cdot 8 + 2 \cdot 5 \end{array} \right| & \left| \begin{array}{l} 1 = 2 \cdot 8 - 3 \cdot 5 \end{array} \right.
 \end{array}$$

Fermat - 2

Sei p eine Primzahl und $1 \leq m < p$, dann gilt:

- ▶ $\text{ggT}(m, p) = 1$ (da p keine Teiler hat)
- ▶ also gibt es a, b mit $1 = \text{ggT}(m, p) = am + bp$
- ▶ also auch $am = 1 - bp$
- ▶ also auch $(am) \bmod p = 1$
- ▶ also aus $m^p \bmod p = m$ folgt $am^p \bmod p = am \bmod p$
- ▶ und schliesslich wegen $am^p = am m^{p-1}$

$$m^{p-1} \bmod p = 1$$

RSA

Wir fixieren zwei Primzahlen: p und q und setzen $N = pq$.

Eine Nachricht m ist eine Zahl $0 \leq m < N$.

- ▶ Wähle $e > 1$ so, dass $\text{ggT}(e, (p-1)(q-1)) = 1$ gilt
- ▶ Berechne f (und k) mit $1 = ef + k(p-1)(q-1)$.

Öffentlich: (N, e) , geheim: f

Verschlüsseln: $m \rightarrow s = m^e \bmod N$

Entschlüsseln: $s \rightarrow s^f \bmod N$

Aber: Warum?

Öffentlich, jeder kennt es:

$$(N, e)$$

damit kann jeder eine Nachricht $0 \leq m < N$ verschlüsseln:

$$m \rightarrow s = m^e \bmod N$$

Geheim, niemand ausser euch:

$$f$$

also könnt ihr (und *nur ihr*)

$$s \rightarrow s^f \bmod N$$

benutzen um die Nachricht zu lesen.

Warum?

Insgesamt:

$$m \rightarrow (m^e \bmod N)^f \bmod N = m^{ef} \bmod N$$

Nach Wahl von f gilt $1 = ef + k(p-1)(q-1)$, also

$$m^{ef} = m^{1-k(p-1)(q-1)} = m \cdot (m^{p-1})^{k(q-1)}$$

p : Falls $m \bmod p = 0$, dann auch $m^{ef} \bmod p = 0 = m \bmod p$.

Sonst: $m^{p-1} \bmod p = (m \bmod p)^{p-1} \bmod p = 1$, also

$$m^{ef} \bmod p = m \cdot (m^{p-1})^{k(q-1)} \bmod p = m \bmod p$$

q : genauso.

Darum!

$$m^{ef} \bmod p = m \bmod p$$

also auch

$$0 = m^{ef} \bmod p - m \bmod p = (m^{ef} - m) \bmod p$$

also p ist ein Teiler von $m^{ef} - m$.

Genauso: q ist auch ein Teiler von $m^{ef} - m$.

Damit ist auch $N = pq$ ein Teiler von $m^{ef} - m$.

Also:

$$m^{ef} \bmod N = m \bmod N = m$$

Beispiel

$p = 11, q = 13$, also

$$N = 143$$

Für $e = 7$ gilt $\text{ggT}(7, 10 \cdot 12) = 1 = -17 \cdot 7 + 1 \cdot 120$ (aber auch
 $1 = (120 - 17) \cdot 7 + (1 - 7) \cdot 120$)

$$e = 7, f = 103$$

Die Nachricht $m = 123$ verschlüsselt sich zu

$$123 \rightarrow (123^7) \bmod 143 = 425927596977747 \bmod 143 = 7$$

Entschlüsseln:

$$7 \rightarrow (7^{103}) \bmod 143 = 110942 \dots 4580343 \bmod 143 = 123$$

(Die Zahl hat 88 Stellen)

Eingesetzt wird dies mit Zahlen die ein bisschen größer sind, *minimal* haben p und q jeweils 160 Stellen, so dass N etwa 320 Dezimalstellen oder 1024 Binärstellen hat.

Empfohlen wird die doppelte oder sogar vierfache *Länge*

(bekannte) Angriffe

- ▶ N zerlegen, also p, q finden (dann f)
- ▶ die selben Primzahlen raten
- ▶ die selbe Nachricht an $> e$ viele Leute schicken
- ▶ falls e „zu klein“ ist, so kann $\sqrt[e]{s} \bmod p$ berechnet werden

Die besten bekannten Methoden N zu zerlegen, konnten eine Zahl mit ca. 240 Stellen in 4.000 CPU Jahren zerlegen. Das ist der Weltrekord.

300 Stellen bräuchten ca. 2.000.000 Jahre, für 600 sind wir bei 540 Milliarden Jahren.

square-and-multiply

oder die russische Bauernmultiplikation...

Falls b gerade ist:

$$a^b = (a^{b/2})^2$$

Sonst:

$$a^b = a(a^{(b-1)/2})^2$$

Zusammen mit mod liefert dies eine effiziente Methode

$$s^f \bmod N$$

zu berechnen. Falls $f \approx N \approx 2^{2048}$ so werden maximal 4096 Multiplikationen (und mods) benutzt.

Falls nach *jeder* Multiplikation reduziert mod wird, so haben die Zahlen maximal 1200 Stellen, was groß ist - aber für den Rechner machbar.

Primzahlen

Um p und q zu finden:

- ▶ Finde eine (ungerade) Zahl der richtigen Größe
- ▶ und teste...

Zum Testen (ob p prim ist):

- ▶ finde ein (zufälliges) $1 < a < p$
- ▶ falls $\text{ggT}(a, p) > 1$, so ist p nicht prim.
- ▶ falls $a^{p-1} \bmod p \neq 1$, so ist p nicht prim.

Wenn dies ein paar mal gelaufen ist, so ist p (sehr) wahrscheinlich prim.

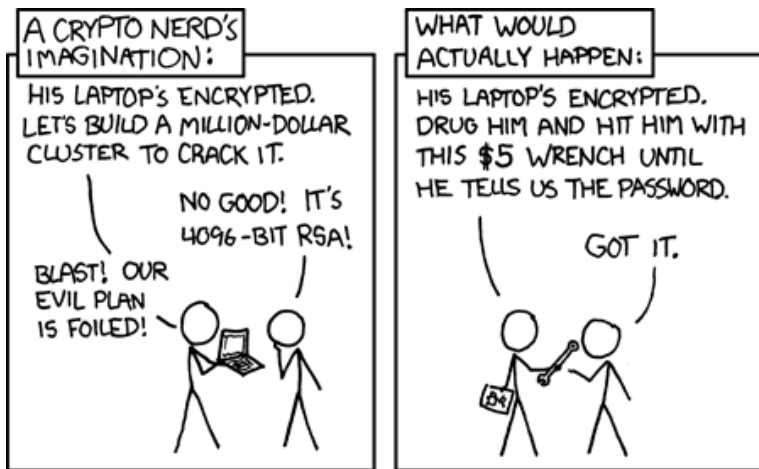
OK: fast, wie müssen den letzten Schritt ändern:

Miller-Rabin

- ▶ $p - 1 = 2^s r$ mit r ungerade.
- ▶ berechne $b = a^r \bmod p$, falls $b = 1$ oder $b = p - 1$, so kann p prim sein.
- ▶ für $i = 1, \dots, s - 1$
- ▶ $b \leftarrow b^2 \bmod p$
- ▶ falls $b = p - 1$, so kann p prim sein
- ▶ p ist *nicht* prim.

Hier ist bewiesen, dass für eine nicht-Primzahl p und ein zufälliges a mit einer Wahrscheinlichkeit von $> 3/4$ die Zahl p als nicht prim erkannt wird.

From <https://xkcd.com/538/>



Python - tools

Input: ein String, Output: eine große Zahl

```
def text_to_num(a):  
    v = 0  
    for i in range(0, len(a)):  
        v = v*256 + ord(a[i])  
    return v
```

... und umgekehrt...

```
def num_to_text(n):  
    a = ""  
    while n > 0:  
        a = chr(n % 256) + a  
        n = n // 256  
    return a
```